

Step 3 - Build and Train an ANN

Beginner Illustrated Click-by-Click Guide

What this guide does

This guide begins at the end of Step 2. It shows every important click, code stage, error-fix, output, evidence file, Git action, and GitHub verification used to complete Step 3.

How to use this guide

SC11 Water Quality Potability is a **worked example** created to show the process. It is not an additional project for students to copy. Follow the same workflow in your own allotted capstone: use your own dataset, problem statement, model files, notebook, Python script, documentation, results, and GitHub repository. The screenshots show SC11 only to demonstrate where to click and what evidence to save.

For SC11, the working files are **02_ann_model.ipynb**, **train_ann.py**, and **STEP-3.md**. In your own project, create equivalent files with suitable names and adapt the ANN/model, features, outputs, and evaluation measures to your project requirements.

Every action is written as a short numbered instruction so beginners can follow it while working on their own project.

Before you start

Step 3 is the core Soft Computing stage. In SC11, the technique is an Artificial Neural Network (ANN). The project uses the clean data created in Step 2 to learn a relationship between nine water-quality values and the potability label.

Expected Step 3 evidence

1. An executed notebook with visible outputs for loading data, training, metrics, graphs, and confusion matrices.
2. One repeatable ANN/model Python script, such as **src/train_ann.py**.
3. A saved loss-curve image from model training.
4. Validation and test metrics recorded clearly.
5. Two labelled confusion matrices: one for validation and one for test data.
6. Prediction CSV files for validation and test data.
7. A saved trained model file and model metadata file.
8. A short Step 3 documentation file explaining the method, results, and limitations.
9. One meaningful Git commit containing the completed Step 3 work.
10. A published GitHub branch for Step 3, ready for review.

Important data rule: Use only **train.csv**, **validation.csv**, and **test.csv** for model development. Generated scenarios are testing scenarios. Do not claim that they are real labelled training data.

Important safety rule: A water-potability prediction is an educational model output. It must not replace laboratory testing or official water-safety certification.

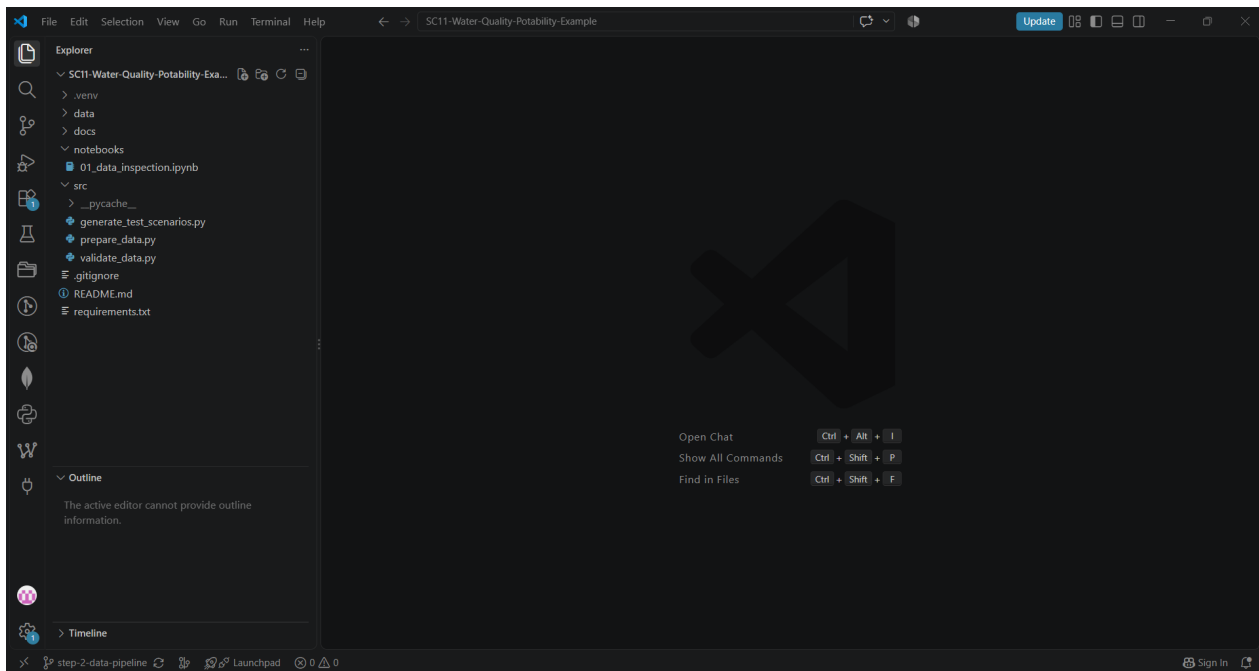
Files created by the end:

1. notebooks/02_ann_model.ipynb - notebook with visible code outputs and graphs.
2. src/train_ann.py - repeatable non-notebook version of the work.
3. results/step3_ann_loss_curve.png and two confusion matrix images.
4. data/reports/step3_ann_metrics.csv plus validation and test prediction files.
5. models/ann_water_potability.pkl and models/ann_model_metadata.json.
6. docs/STEP-3.md and a pushed step-3-ann-model branch.

Action 1: Open the completed Step 2 project

Start from the local folder that already contains your finished Step 2 work.

1. Open VS Code and select **File -> Open Folder** if the project is not already open.
2. Choose the existing SC11 project folder. Do not clone it again on the same computer.
3. Look at the bottom-left status bar. It should show **step-2-data-pipeline** before you create Step 3.
4. In Explorer, confirm that the folders **data**, **docs**, **notebooks**, and **src** are visible.

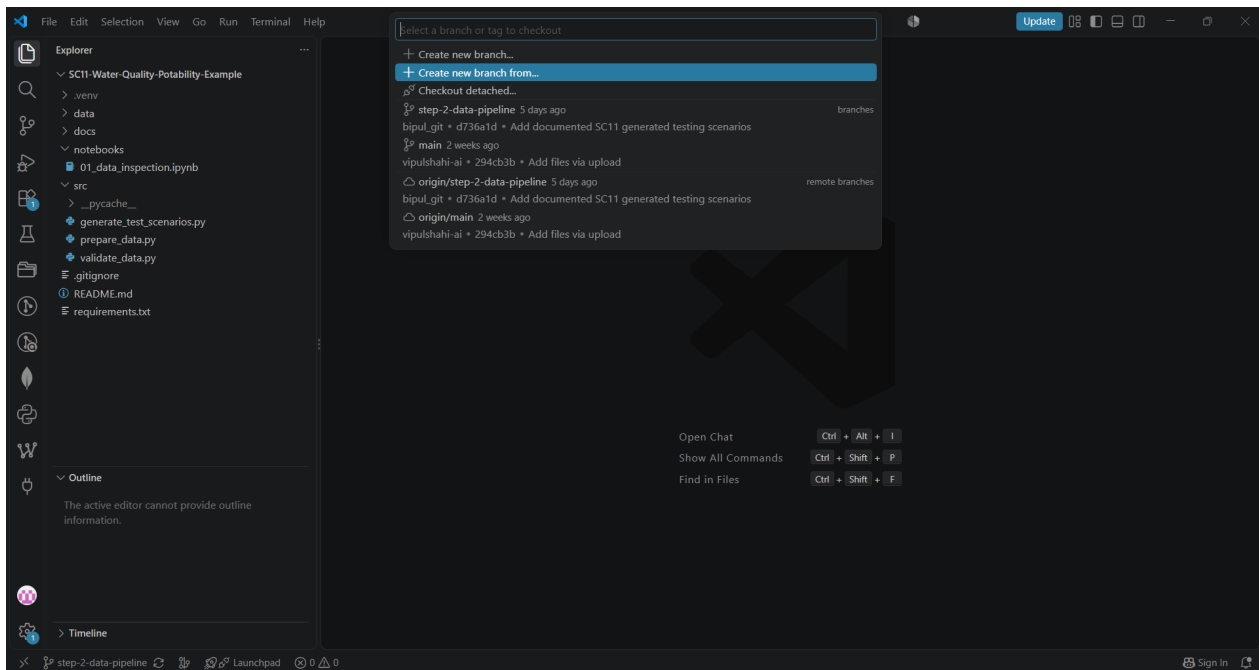


Screenshot 1. Completed Step 2 project open in VS Code.

Action 2: Choose the correct branch command

A branch keeps Step 3 work separate from Step 2 and main.

1. Click the branch name at the bottom-left of VS Code.
2. Choose **Create new branch from...** - not just Create new branch.
3. This choice lets you explicitly select Step 2 as the starting point.

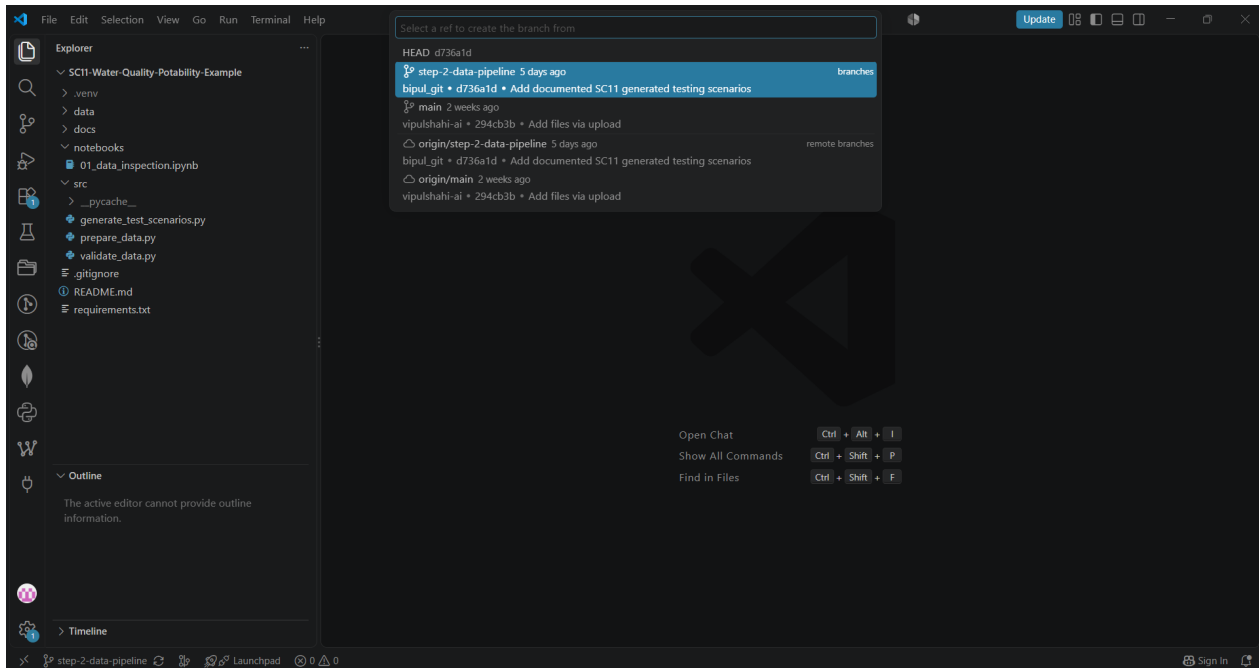


Screenshot 2. Select Create new branch from...

Action 3: Use Step 2 as the starting point

The new Step 3 branch must inherit the completed data pipeline.

1. In the list, click **step-2-data-pipeline** under local branches.
2. Do not choose main, because main does not contain the completed Step 2 work.
3. Confirm that the selected Step 2 branch displays the latest Step 2 commit message.

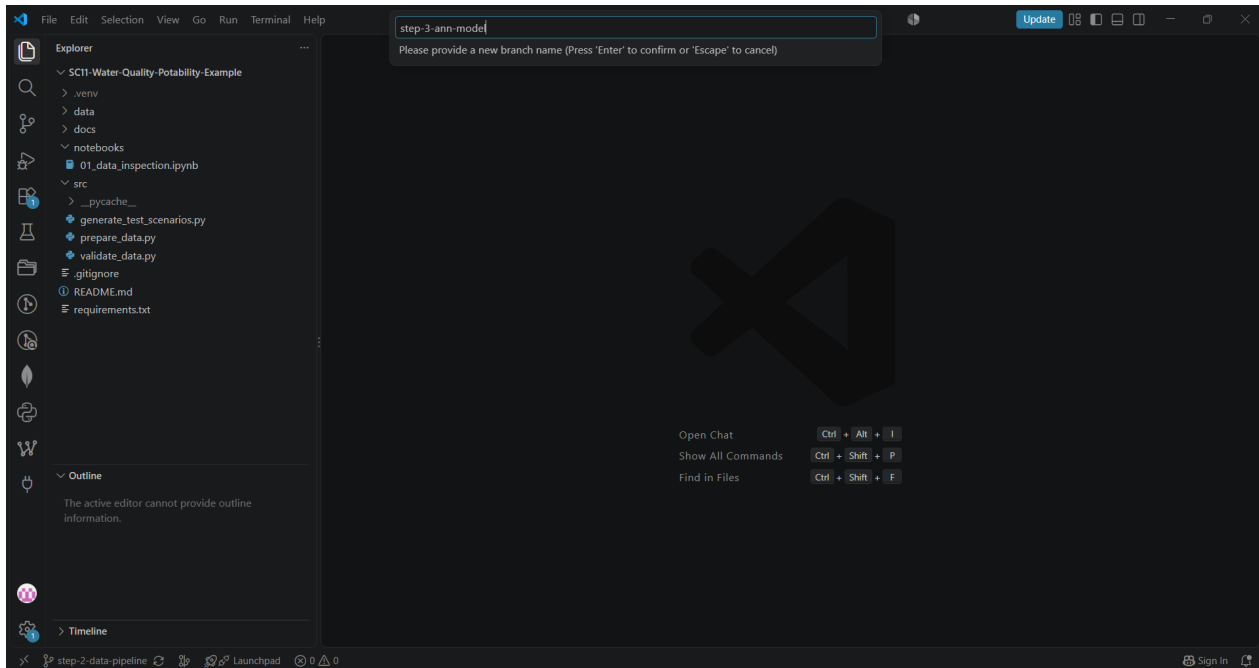


Screenshot 3. Choose step-2-data-pipeline as the source branch.

Action 4: Name the Step 3 branch

Use a clear, standard branch name so your instructor can identify the work easily.

1. Type exactly **step-3-ann-model**.
2. Press Enter to create the branch.
3. Check the bottom-left status bar once more. It must now show **step-3-ann-model**.

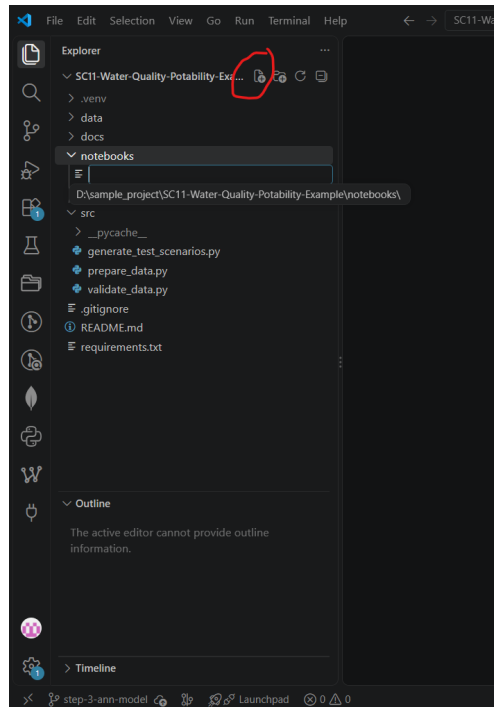


Screenshot 4. Enter the exact Step 3 branch name.

Action 5: Create the ANN notebook

A notebook keeps explanations, code, results, tables, and graphs together for easy review.

1. Expand the **notebooks** folder in Explorer.
2. Click the New File icon beside Explorer or right-click notebooks and choose New File.
3. Create the notebook inside notebooks, not at the project root.

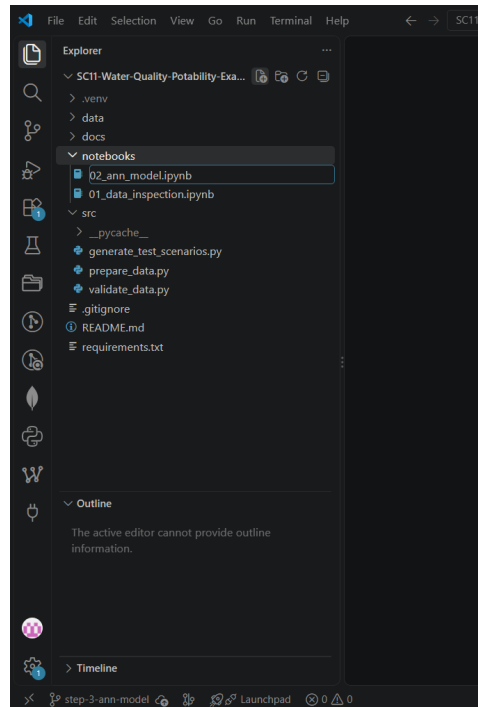


Screenshot 5. The circled icon creates a new file inside the notebooks folder.

Action 6: Give the notebook the correct name

A consistent name makes the file easy to find during checking and Git review.

1. Type **02_ann_model.ipynb** exactly.
2. Press Enter and wait for VS Code to open the notebook tab.
3. Keep the earlier **01_data_inspection.ipynb**; it is Step 2 evidence and should not be replaced.

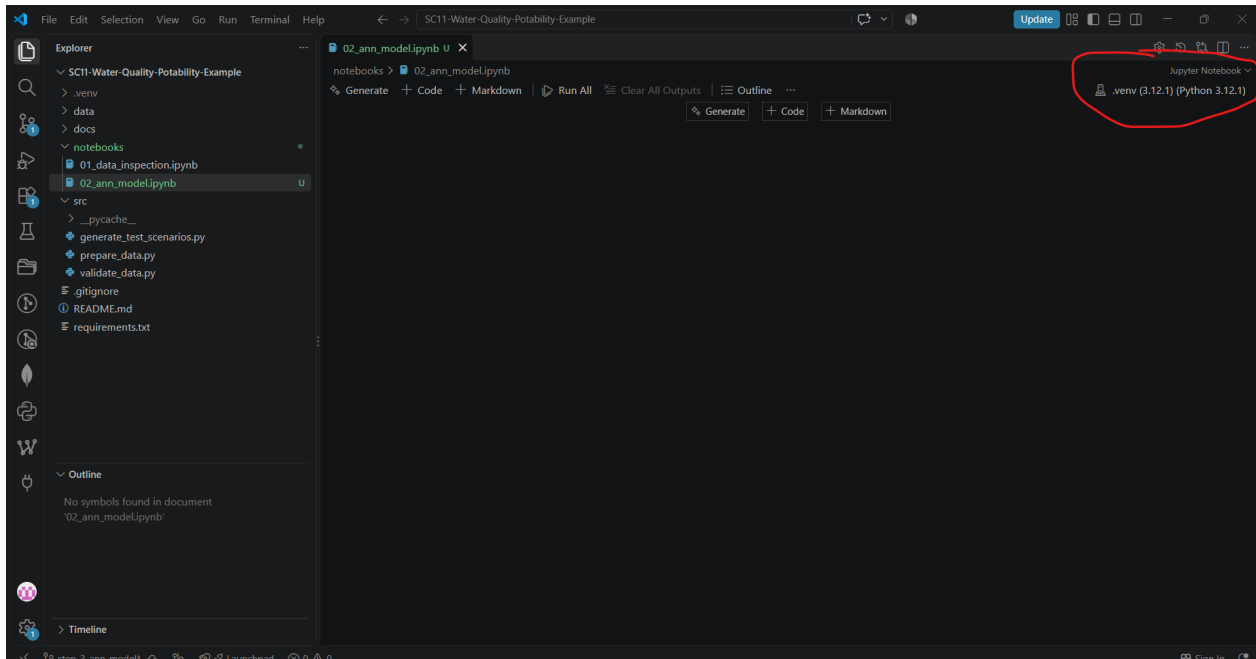


Screenshot 6. *02_ann_model.ipynb* is created below *01_data_inspection.ipynb*.

Action 7: Select the Python kernel

The kernel is the Python environment that runs notebook cells.

1. Open the notebook and look at the top-right corner.
2. Click the kernel name and choose the project virtual environment, usually **.venv**.
3. Use the same environment throughout Step 3 so installed packages and results remain consistent.

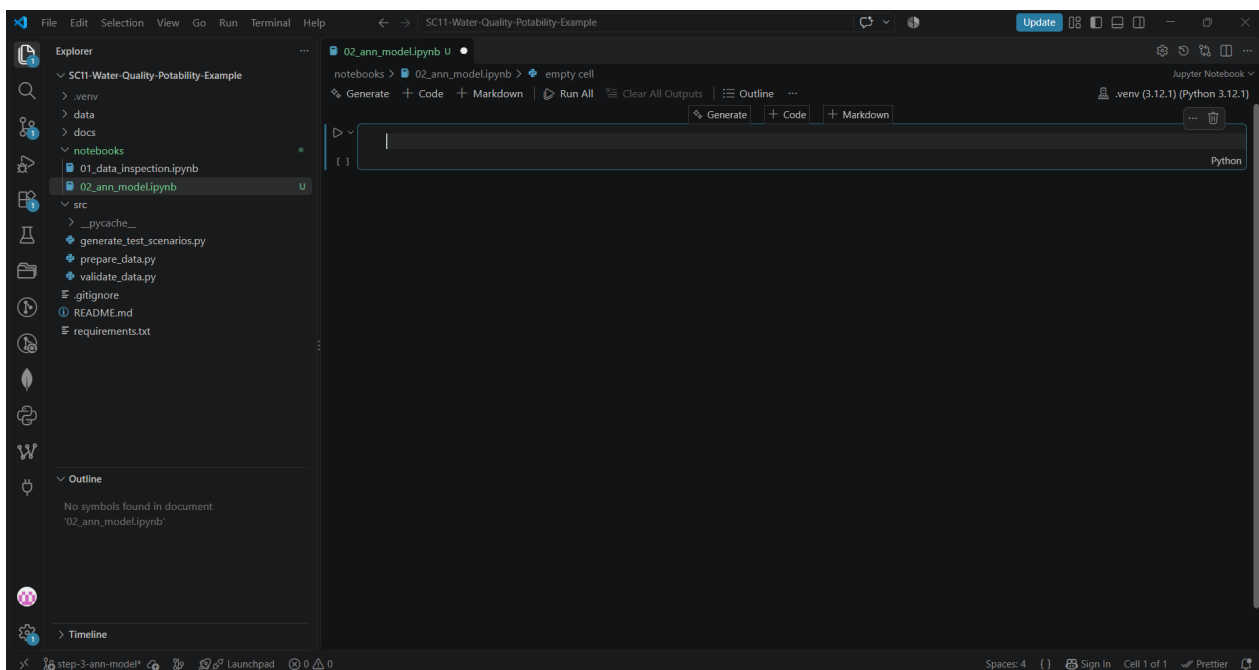


Screenshot 7. The selected **.venv** Python kernel is highlighted.

Action 8: Create and adapt the first notebook cell

The first cell imports libraries, defines project folders, and lists the inputs for the project model.

1. Use the SC11 example notebook only as a structural reference; do not copy it unchanged into your own project.
2. Create the first code cell and write/adapt the imports, folder paths, and feature names for your own dataset and project.
3. Use the same idea of clear folder variables such as **PROJECT_ROOT**, **DATA_DIR**, **REPORT_DIR**, **MODEL_DIR**, and **RESULTS_DIR** where appropriate.
4. Click the triangular Run button on the left after you have written and checked your code.



Screenshot 8. First cell: imports, folders, feature names, and output folders.

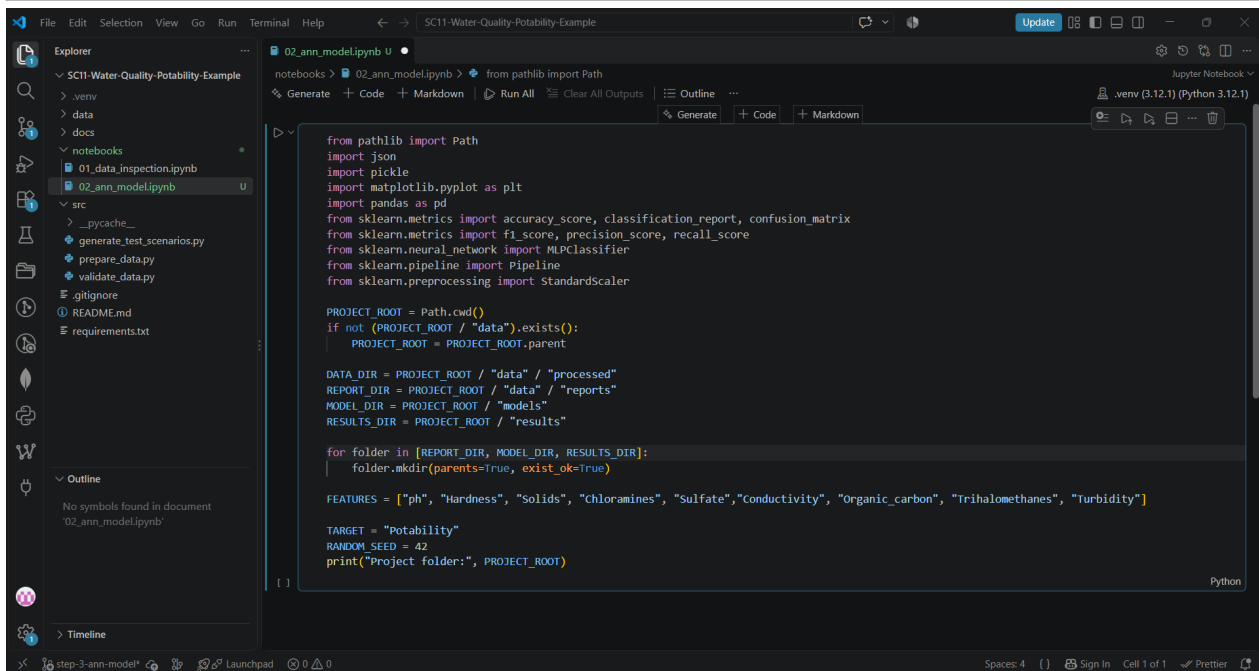
Action 9: Read the first package error calmly

A missing-package error is common on a new Python environment. It does not mean the project code is wrong.

1. Read the last line of the error message. Here it says **No module named matplotlib**.
2. Do not delete the cell or change the model code.
3. Install the missing package in the selected project environment, then rerun the same cell.

Why this matters

Always fix the named package first. Rerun the same cell only after installation completes.



The screenshot shows a Jupyter Notebook interface within a VS Code editor. The notebook is titled '02_ann_model.ipynb'. The code cell contains imports for various libraries, including 'matplotlib.pyplot as plt'. The error message at the bottom of the cell states: 'ImportError: No module named matplotlib'. The file explorer on the left shows the project structure, including a 'notebooks' folder with '01_data_inspection.ipynb' and '02_ann_model.ipynb'. The terminal at the bottom shows the command 'step-3-ann-model'.

```
from pathlib import Path
import json
import pickle
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import f1_score, precision_score, recall_score
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

PROJECT_ROOT = Path.cwd()
if not (PROJECT_ROOT / "data").exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_DIR = PROJECT_ROOT / "data" / "processed"
REPORT_DIR = PROJECT_ROOT / "data" / "reports"
MODEL_DIR = PROJECT_ROOT / "models"
RESULTS_DIR = PROJECT_ROOT / "results"

for folder in [REPORT_DIR, MODEL_DIR, RESULTS_DIR]:
    folder.mkdir(parents=True, exist_ok=True)

FEATURES = ["ph", "Hardness", "Solids", "Chloramines", "Sulfate", "Conductivity", "Organic_carbon", "Trihalomethanes", "Turbidity"]

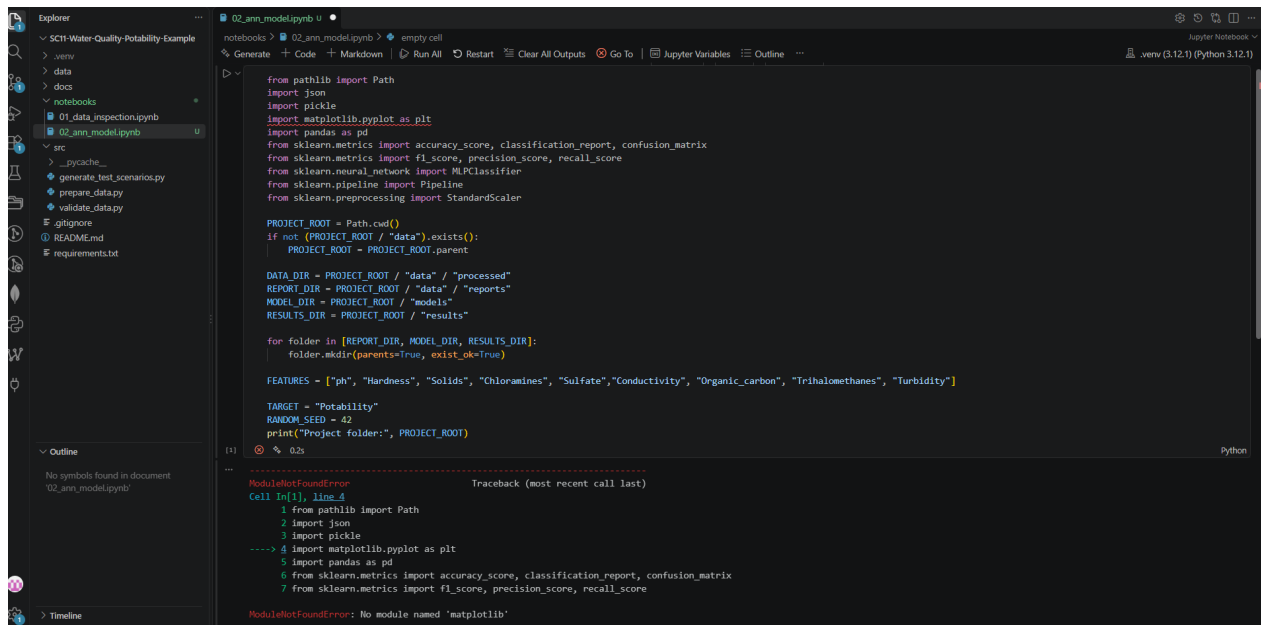
TARGET = "Potability"
RANDOM_SEED = 42
print("Project folder:", PROJECT_ROOT)
```

Screenshot 9. The error identifies matplotlib as the missing dependency.

Action 10: Install the missing package

Use the VS Code terminal only for the dependency installation shown here.

1. Open **Terminal** -> **New Terminal** in VS Code.
2. Check that the prompt begins with **(.venv)**. This confirms the correct environment is active.
3. Type **pip install matplotlib** and wait until installation completes.
4. Return to the notebook and run the first cell again.



The screenshot shows the VS Code interface with a Jupyter Notebook open. The notebook contains Python code for a water quality potability model. The code imports various libraries including pathlib, json, pickle, matplotlib.pyplot as plt, pandas as pd, and sklearn metrics and preprocessing modules. It defines project directories and features, and sets a target variable. The code is executed in a cell, but it fails with a `ModuleNotFoundError: No module named 'matplotlib'`. Below the notebook, a terminal window is open, showing the prompt `(.venv) %` and the command `0.2s`, indicating the environment is active and the command is being executed.

```
from pathlib import Path
import json
import pickle
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import f1_score, precision_score, recall_score
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

PROJECT_ROOT = Path.cwd()
if not (PROJECT_ROOT / "data").exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_DIR = PROJECT_ROOT / "data" / "processed"
REPORT_DIR = PROJECT_ROOT / "data" / "reports"
MODEL_DIR = PROJECT_ROOT / "models"
RESULTS_DIR = PROJECT_ROOT / "results"

for folder in [REPORT_DIR, MODEL_DIR, RESULTS_DIR]:
    folder.mkdir(parents=True, exist_ok=True)

FEATURES = ["ph", "Hardness", "Solids", "Chloramines", "Sulfate", "Conductivity", "Organic_carbon", "Trihalomethanes", "Turbidity"]

TARGET = "Potability"
RANDOM_SEED = 42
print("Project folder:", PROJECT_ROOT)
```

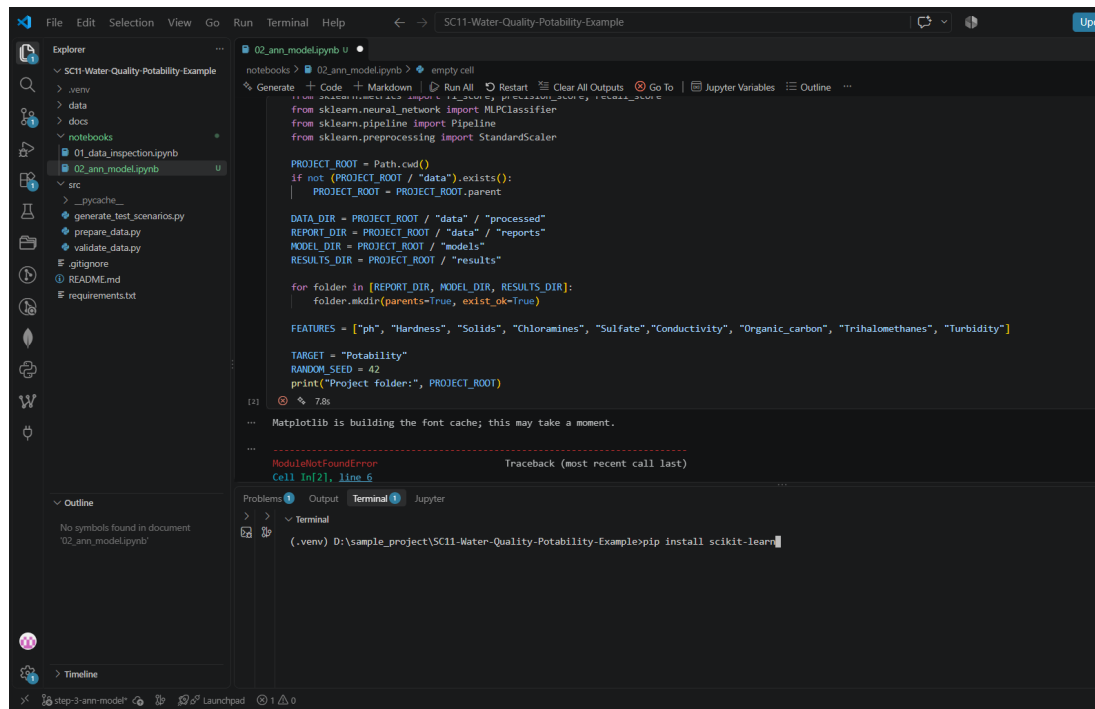
```
ModuleNotFoundError: No module named 'matplotlib'
```

Screenshot 10. Install matplotlib in the active .venv environment.

Action 11: Install scikit-learn if it is also missing

The ANN model comes from scikit-learn, so this package is required for Step 3.

1. If the rerun reports **No module named sklearn**, open the terminal again.
2. Type **pip install scikit-learn** and wait for a successful completion message.
3. Do not install packages in a different Python window or global environment by mistake.

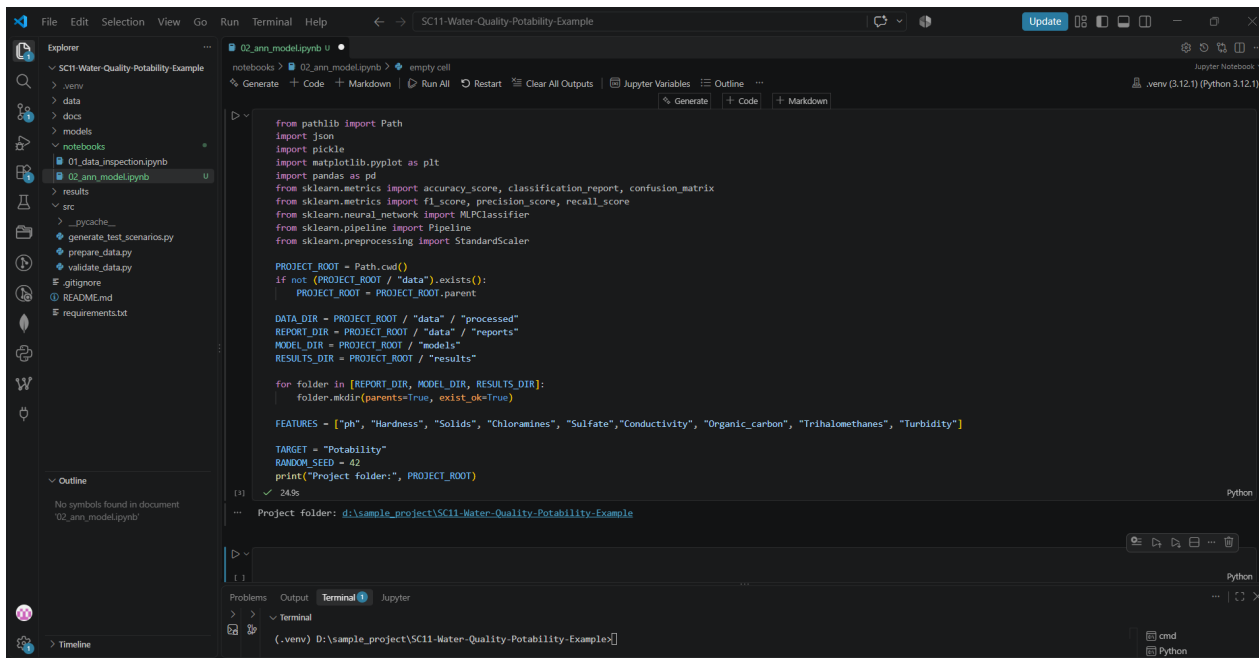


Screenshot 11. Install scikit-learn only after the error names it.

Action 12: Run the first cell successfully

The first successful output proves that the notebook can find the project location and required libraries.

1. Return to the notebook after all required packages are installed.
2. Run the first cell again. A green tick means it completed successfully.
3. Check that the printed project folder is the SC11 repository folder, not an unrelated folder.



```
from pathlib import Path
import json
import pickle
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.metrics import f1_score, precision_score, recall_score
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

PROJECT_ROOT = Path.cwd()
if not (PROJECT_ROOT / "data").exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_DIR = PROJECT_ROOT / "data" / "processed"
REPORT_DIR = PROJECT_ROOT / "data" / "reports"
MODEL_DIR = PROJECT_ROOT / "models"
RESULTS_DIR = PROJECT_ROOT / "results"

for folder in [REPORT_DIR, MODEL_DIR, RESULTS_DIR]:
    folder.mkdir(parents=True, exist_ok=True)

FEATURES = ["ph", "Hardness", "Solids", "Chloramines", "Sulfate", "Conductivity", "Organic_carbon", "Trihalomethanes", "Turbidity"]

TARGET = "Potability"
RANDOM_SEED = 42
print("Project folder:", PROJECT_ROOT)
```

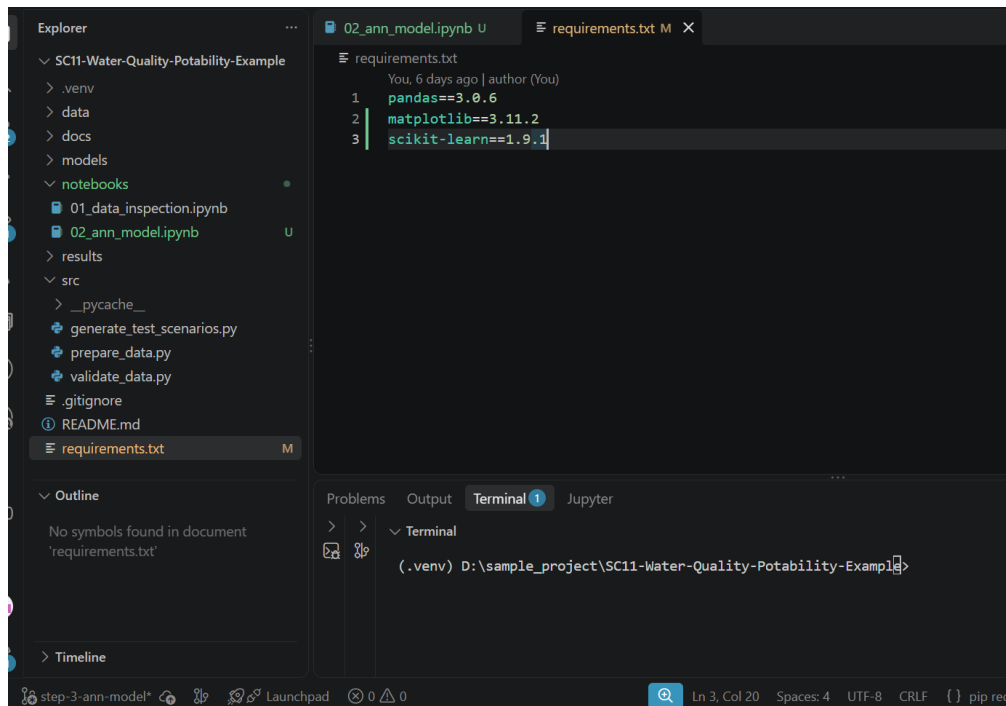
Project folder: d:\sample_project\SC11-Water-Quality-Potability-Example

Screenshot 12. Successful first cell: VS Code shows the project folder.

Action 13: Record reproducible dependencies

requirements.txt allows a teammate or evaluator to recreate the same environment.

1. Open **requirements.txt** in Explorer.
2. Keep pandas and add matplotlib and scikit-learn as shown.
3. Save the file. It will appear as modified in Source Control later.

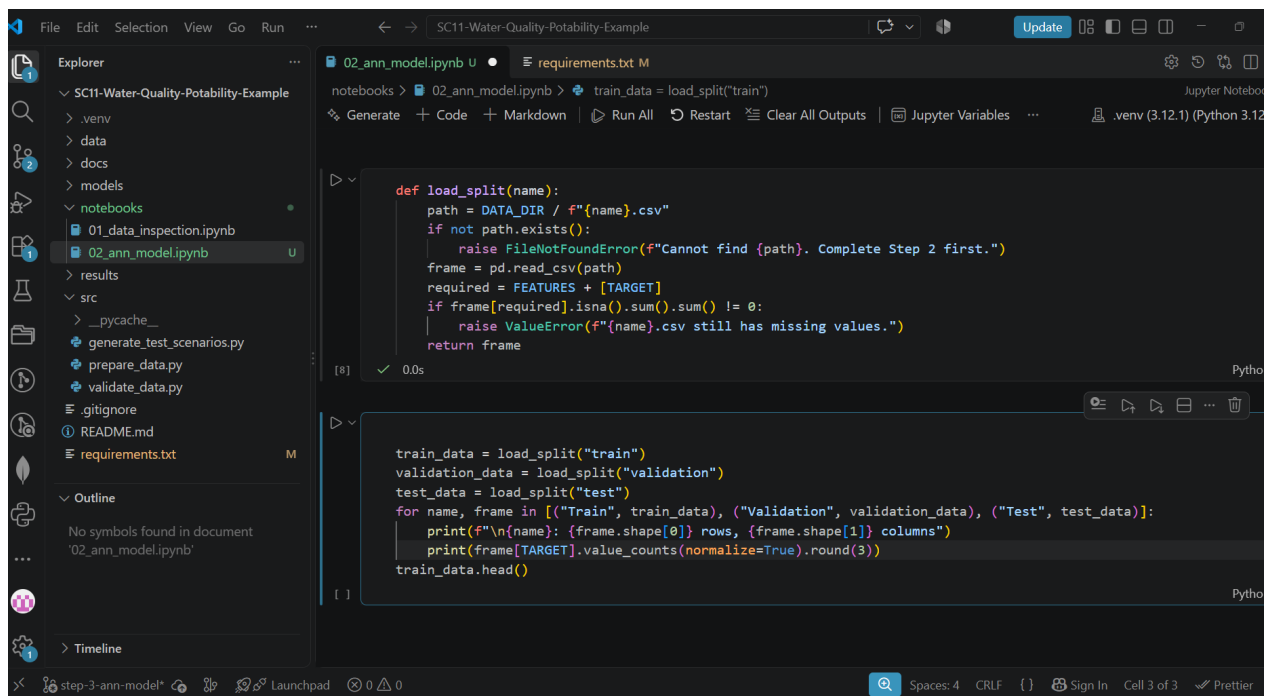


Screenshot 13. requirements.txt now records pandas, matplotlib, and scikit-learn.

Action 14: Write or adapt a data-loading function

This function ensures each required Step 2 split exists and has no missing values before training begins.

1. Use the SC11 `load_split` function only as an idea. Write or adapt a function for your own filenames, columns, and data folders.
2. For SC11 it checks `train.csv`, `validation.csv`, and `test.csv` in `data/processed`. Your project may use different files, but should keep the same clear validation idea.
3. Run the function cell. A green tick confirms the function is ready but does not load data yet.



The screenshot shows a Jupyter Notebook environment with a dark theme. The left sidebar displays the file explorer for a project named 'SC11-Water-Quality-Potability-Example'. The main area contains two code cells. The first cell defines the `load_split` function, which takes a name, constructs a path, checks for file existence, reads the CSV, and validates required columns. The second cell demonstrates the function's use by loading 'train', 'validation', and 'test' data, printing their shapes and column counts, and displaying the first rows of the training data.

```
def load_split(name):
    path = DATA_DIR / f"{name}.csv"
    if not path.exists():
        raise FileNotFoundError(f"Cannot find {path}. Complete Step 2 first.")
    frame = pd.read_csv(path)
    required = FEATURES + [TARGET]
    if frame[required].isna().sum().sum() != 0:
        raise ValueError(f"{name}.csv still has missing values.")
    return frame

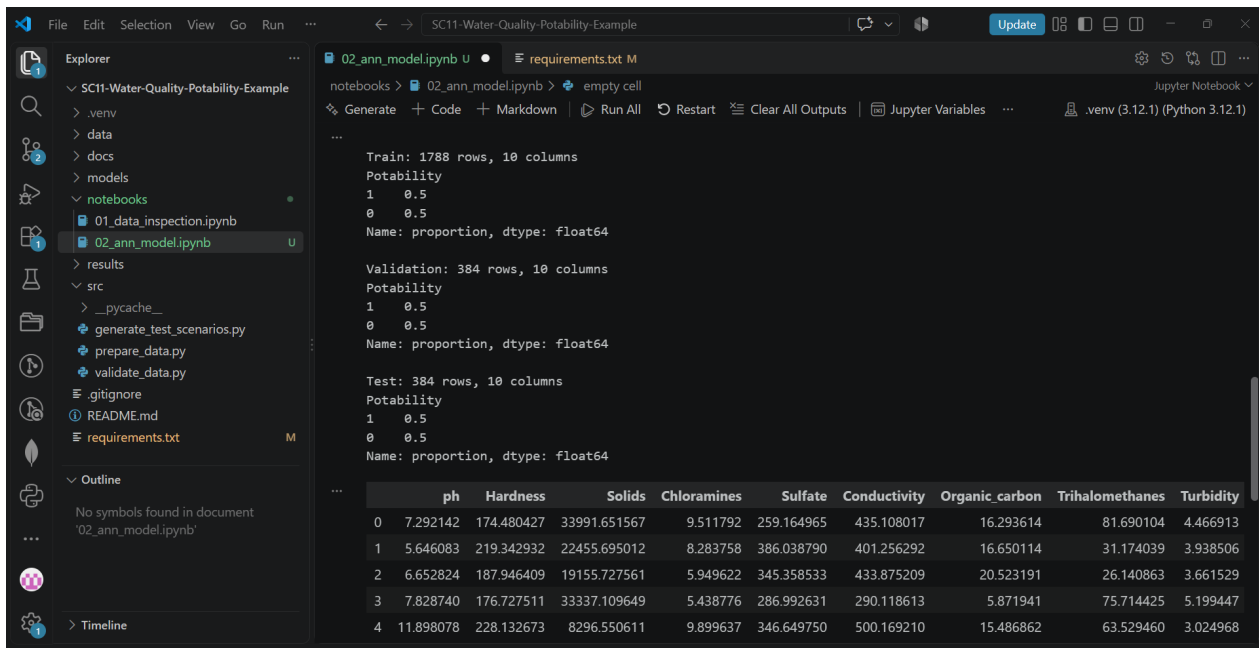
train_data = load_split("train")
validation_data = load_split("validation")
test_data = load_split("test")
for name, frame in [("Train", train_data), ("Validation", validation_data), ("Test", test_data)]:
    print(f"\n{name}: {frame.shape[0]} rows, {frame.shape[1]} columns")
    print(frame[TARGET].value_counts(normalize=True).round(3))
train_data.head()
```

Screenshot 14. The `load_split` function validates each Step 2 input file.

Action 15: Load and inspect train, validation, and test data

This is the proof that the model uses prepared Step 2 data rather than raw data.

1. Run the next cell to load all three CSV files.
2. Check row counts: training data is largest, while validation and test are separate.
3. Check class proportions. Here both labels are balanced at 0.5 in every split.
4. Keep the displayed data preview visible; it is useful evidence for the guide and viva.



The screenshot shows a Jupyter Notebook interface with the following content:

```
...
Train: 1788 rows, 10 columns
Potability
1    0.5
0    0.5
Name: proportion, dtype: float64

Validation: 384 rows, 10 columns
Potability
1    0.5
0    0.5
Name: proportion, dtype: float64

Test: 384 rows, 10 columns
Potability
1    0.5
0    0.5
Name: proportion, dtype: float64
...
```

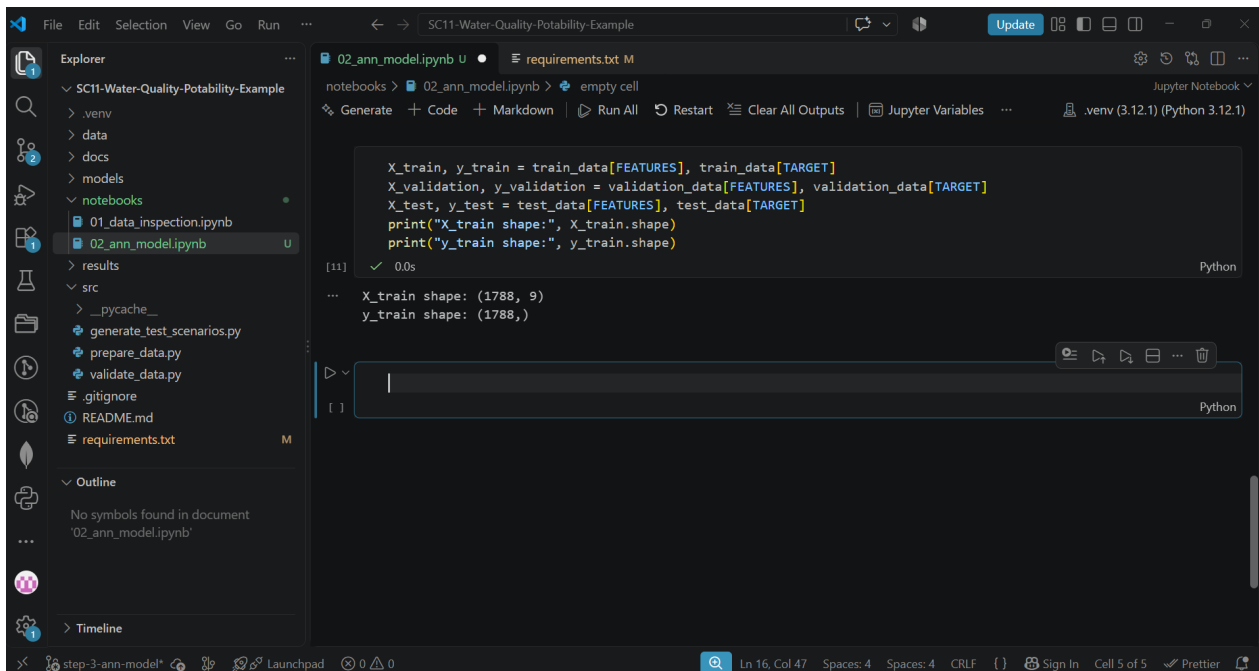
	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity
0	7.292142	174.480427	33991.651567	9.511792	259.164965	435.108017	16.293614	81.690104	4.466913
1	5.646083	219.342932	22455.695012	8.283758	386.038790	401.256292	16.650114	31.174039	3.938506
2	6.652824	187.946409	19155.727561	5.949622	345.358533	433.875209	20.523191	26.140863	3.661529
3	7.828740	176.727511	33337.109649	5.438776	286.992631	290.118613	5.871941	75.714425	5.199447
4	11.898078	228.132673	8296.550611	9.899637	346.649750	500.169210	15.486862	63.529460	3.024968

Screenshot 15. Split sizes, class proportions, and a preview of the cleaned training data.

Action 16: Separate inputs from the answer

The ANN should receive only water measurements as inputs and the known Potability column as the answer to learn.

1. Set **X_train** to the nine feature columns and **y_train** to Potability.
2. Do the same for validation and test data.
3. Run the cell and confirm X_train shows **(1788, 9)**: 1788 rows and nine inputs.



```
X_train, y_train = train_data[FEATURES], train_data[TARGET]
X_validation, y_validation = validation_data[FEATURES], validation_data[TARGET]
X_test, y_test = test_data[FEATURES], test_data[TARGET]
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
```

```
[11] ✓ 0.0s
```

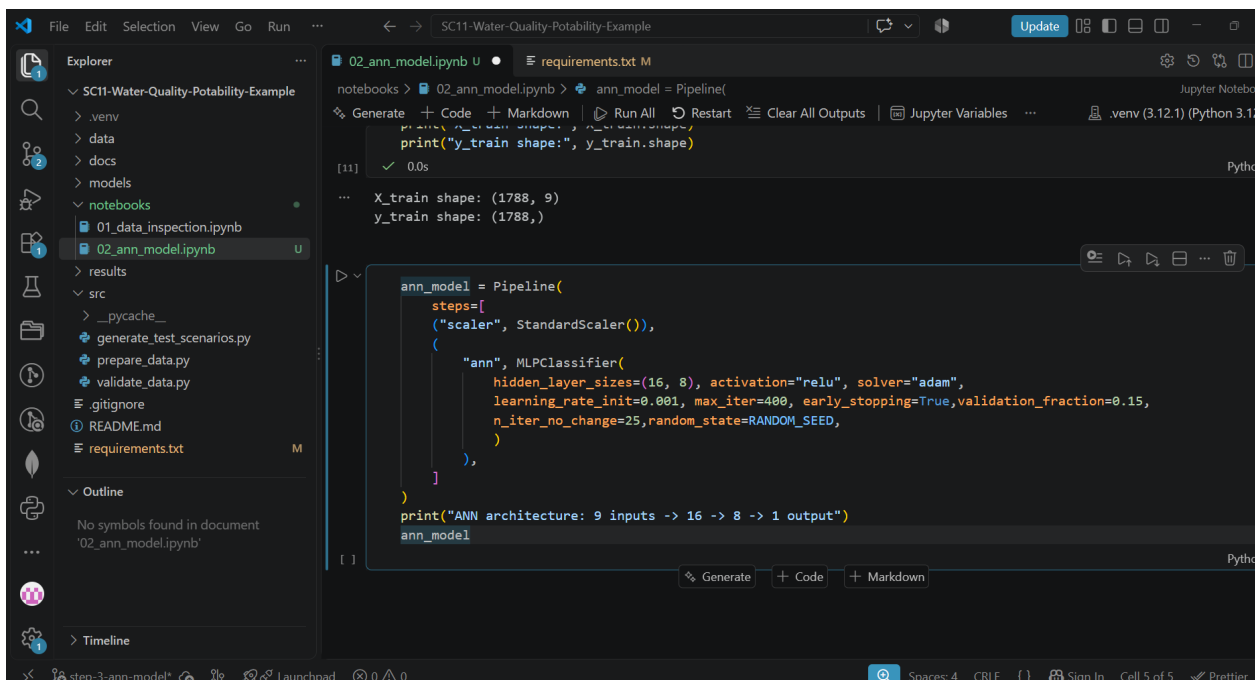
```
... X_train shape: (1788, 9)
y_train shape: (1788,)
```

Screenshot 16. X_train has nine model-input columns; y_train holds one output label.

Action 17: Build an ANN pipeline for your project

The pipeline standardizes data first and then trains a feed-forward ANN.

1. Use the SC11 pipeline as a reference, then write/adapt an ANN pipeline that matches your own project inputs and output label.
2. The SC11 architecture is **9 inputs -> 16 hidden neurons -> 8 hidden neurons -> 1 output**. Select and justify an architecture suitable for your own project instead of copying these numbers automatically.
3. Use a fixed random seed, such as 42, so your result can be reproduced.
4. Run the cell once to create the model configuration; it does not train the model yet.



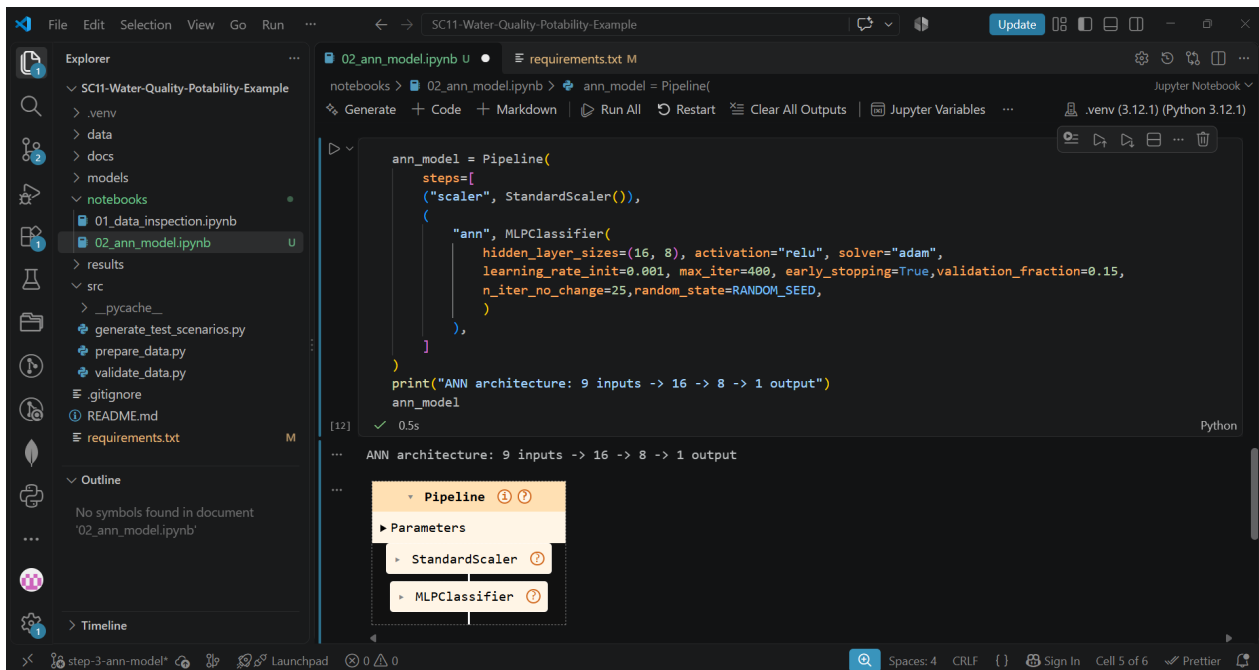
```
notebooks > 02_ann_model.ipynb > ann_model = Pipeline(  
    steps=[  
        ("scaler", StandardScaler()),  
        ("ann", MLPClassifier(  
            hidden_layer_sizes=(16, 8), activation="relu", solver="adam",  
            learning_rate_init=0.001, max_iter=400, early_stopping=True, validation_fraction=0.15,  
            n_iter_no_change=25, random_state=42,  
        ))  
    ],  
)  
print("ANN architecture: 9 inputs -> 16 -> 8 -> 1 output")  
ann_model
```

Screenshot 17. ANN configuration with scaling, two hidden layers, Adam optimizer, and early stopping.

Action 18: Read the pipeline output

The displayed pipeline is evidence that the model includes both feature scaling and an MLPClassifier ANN.

1. Check that the output displays **StandardScaler** followed by **MLPClassifier**.
2. The scaler prevents features with very large numerical ranges from dominating the learning process.
3. The MLPClassifier is the feed-forward ANN trained using backpropagation.



The screenshot shows a Jupyter Notebook interface with a dark theme. The left sidebar displays a file explorer for a project named 'SC11-Water-Quality-Potability-Example', with files like '.venv', 'data', 'docs', 'models', 'notebooks', and 'requirements.txt'. The main area shows a code cell with the following Python code:

```
ann_model = Pipeline(  
    steps=[  
        ("scaler", StandardScaler()),  
        (  
            "ann", MLPClassifier(  
                hidden_layer_sizes=(16, 8), activation="relu", solver="adam",  
                learning_rate_init=0.001, max_iter=400, early_stopping=True, validation_fraction=0.15,  
                n_iter_no_change=25, random_state=RANDOM_SEED,  
            )  
        ),  
    ]  
)  
print("ANN architecture: 9 inputs -> 16 -> 8 -> 1 output")  
ann_model
```

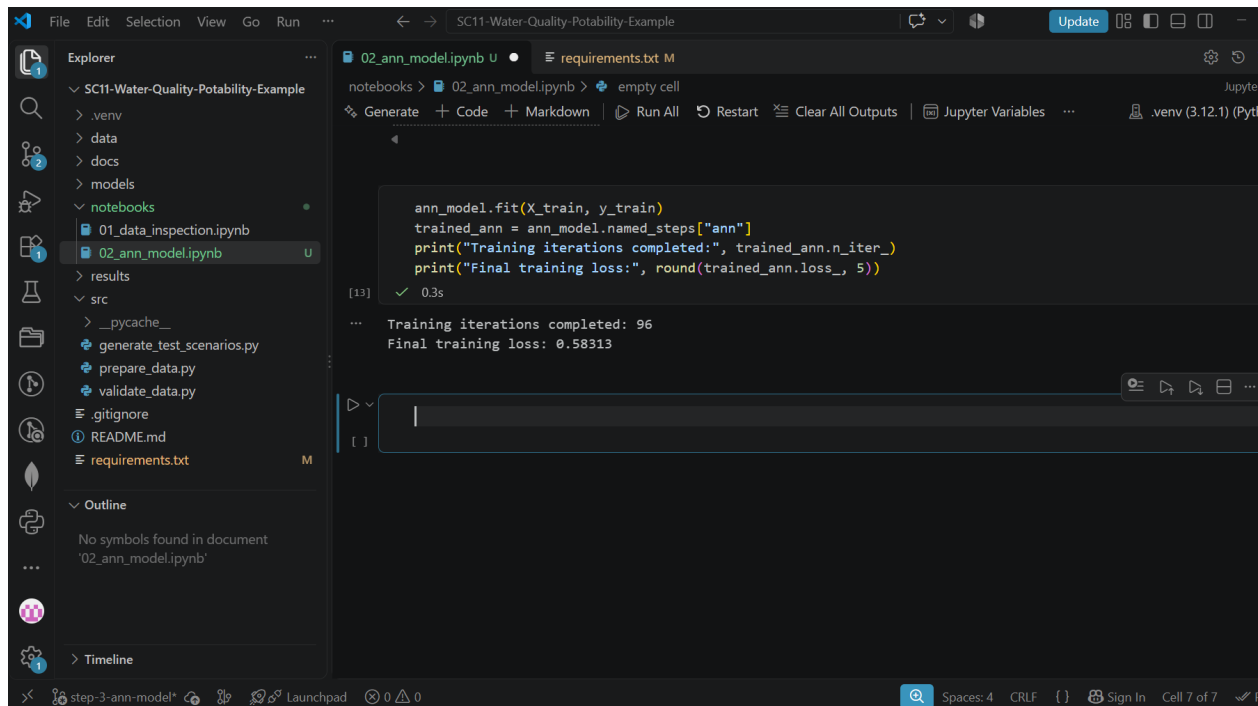
Below the code, the output is displayed: 'ANN architecture: 9 inputs -> 16 -> 8 -> 1 output'. A visual representation of the pipeline is shown as a diagram with a 'Pipeline' box containing 'StandardScaler' and 'MLPClassifier' steps.

Screenshot 18. Pipeline output confirms the scaler and ANN are connected.

Action 19: Train the ANN

Training lets the ANN compare its predictions with known training labels and adjust its internal weights.

1. Run the training cell containing `ann_model.fit(X_train, y_train)`.
2. Wait for the cell to finish. Do not click Run repeatedly while it is working.
3. Record both the number of training iterations and the final loss value.
4. Here the model completed 96 iterations and reached a loss of 0.58313.

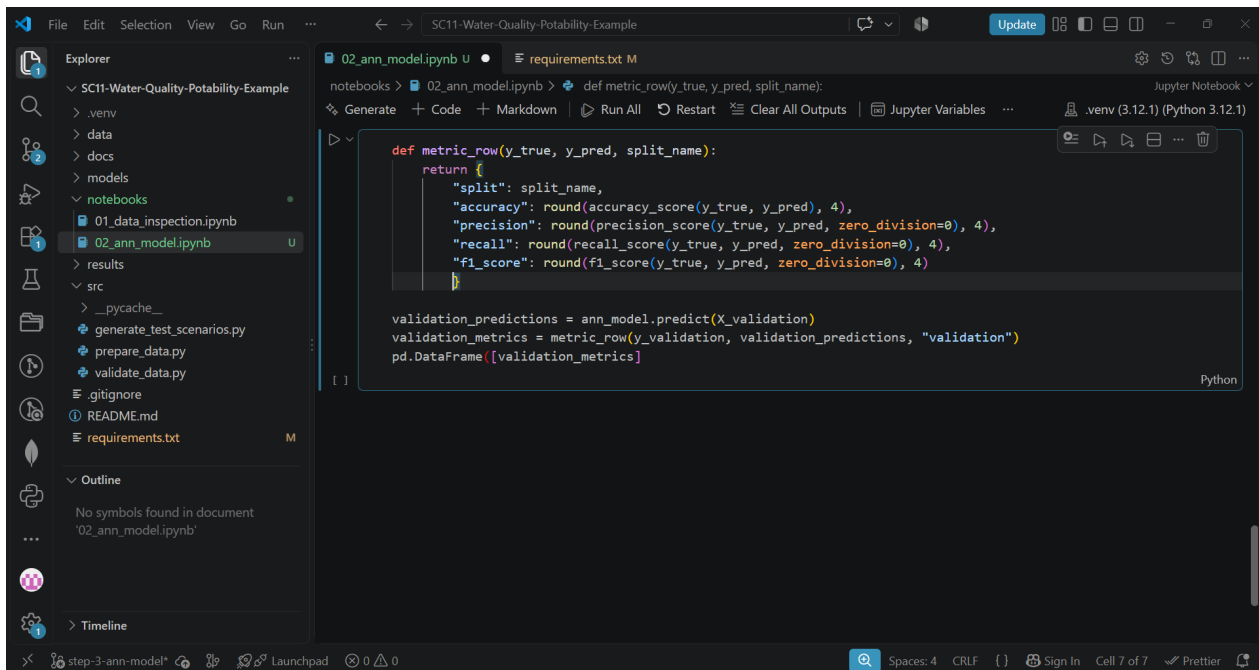


Screenshot 19. Training completes and reports the number of iterations and final loss.

Action 20: Create validation metrics

Validation data checks the model during development before the final test set is interpreted.

1. Use the SC11 **metric_row** function as a reference and write/adapt your own evaluation function.
2. It calculates accuracy, precision, recall, and F1-score.
3. Run the cell to generate predictions from `X_validation` and display the metric table.



The screenshot shows a Jupyter Notebook environment with a dark theme. The left sidebar displays the file explorer for a project named 'SC11-Water-Quality-Potability-Example'. The main area shows a code cell in '02_ann_model.ipynb' with the following Python code:

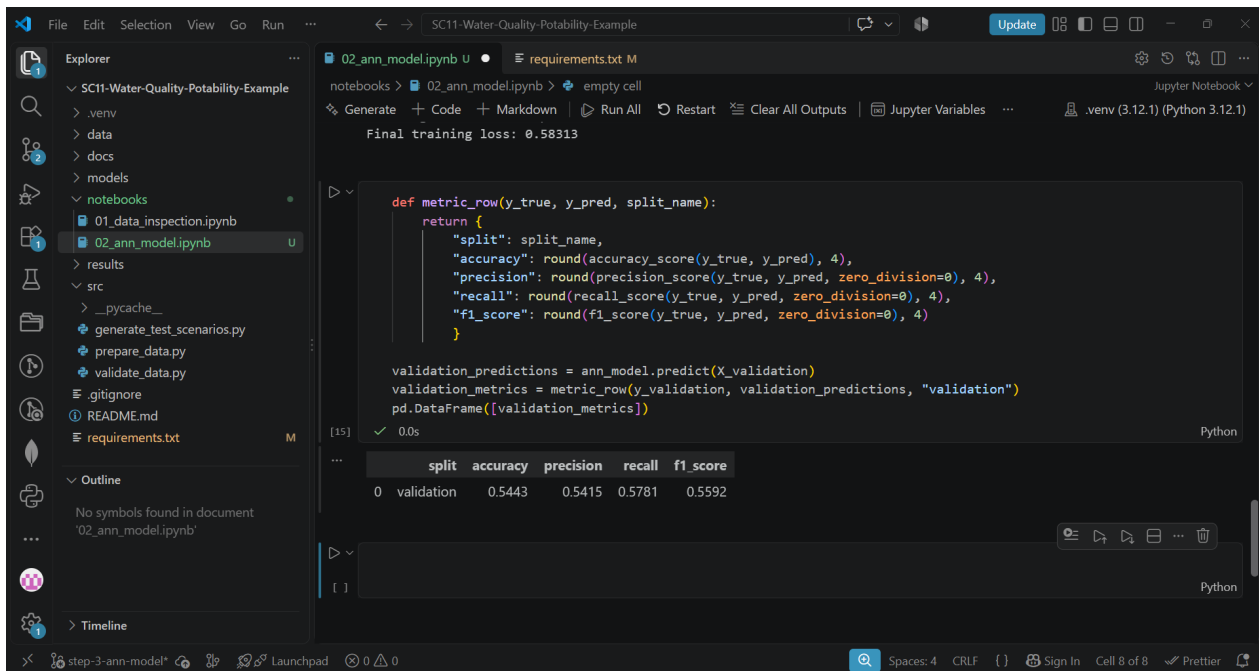
```
def metric_row(y_true, y_pred, split_name):  
    return {  
        "split": split_name,  
        "accuracy": round(accuracy_score(y_true, y_pred), 4),  
        "precision": round(precision_score(y_true, y_pred, zero_division=0), 4),  
        "recall": round(recall_score(y_true, y_pred, zero_division=0), 4),  
        "f1_score": round(f1_score(y_true, y_pred, zero_division=0), 4)  
    }  
  
validation_predictions = ann_model.predict(X_validation)  
validation_metrics = metric_row(y_validation, validation_predictions, "validation")  
pd.DataFrame([validation_metrics])
```

Screenshot 20. Validation metric function and prediction code.

Action 21: Record the validation result

One number is not enough. Record all four metrics so you can discuss the model properly in viva.

1. Check that the displayed row is labelled **validation**.
2. Accuracy is total correct predictions; precision checks the reliability of positive predictions.
3. Recall checks how many actual positive cases were found; F1 balances precision and recall.
4. Do not edit a low value. Honest results are the starting point for later improvement.



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'SC11-Water-Quality-Potability-Example' with various files and folders. The code editor shows a Python cell with the following code:

```
def metric_row(y_true, y_pred, split_name):  
    return {  
        "split": split_name,  
        "accuracy": round(accuracy_score(y_true, y_pred), 4),  
        "precision": round(precision_score(y_true, y_pred, zero_division=0), 4),  
        "recall": round(recall_score(y_true, y_pred, zero_division=0), 4),  
        "f1_score": round(f1_score(y_true, y_pred, zero_division=0), 4)  
    }  
  
validation_predictions = ann_model.predict(X_validation)  
validation_metrics = metric_row(y_validation, validation_predictions, "validation")  
pd.DataFrame([validation_metrics])
```

The output of the code is a table with the following data:

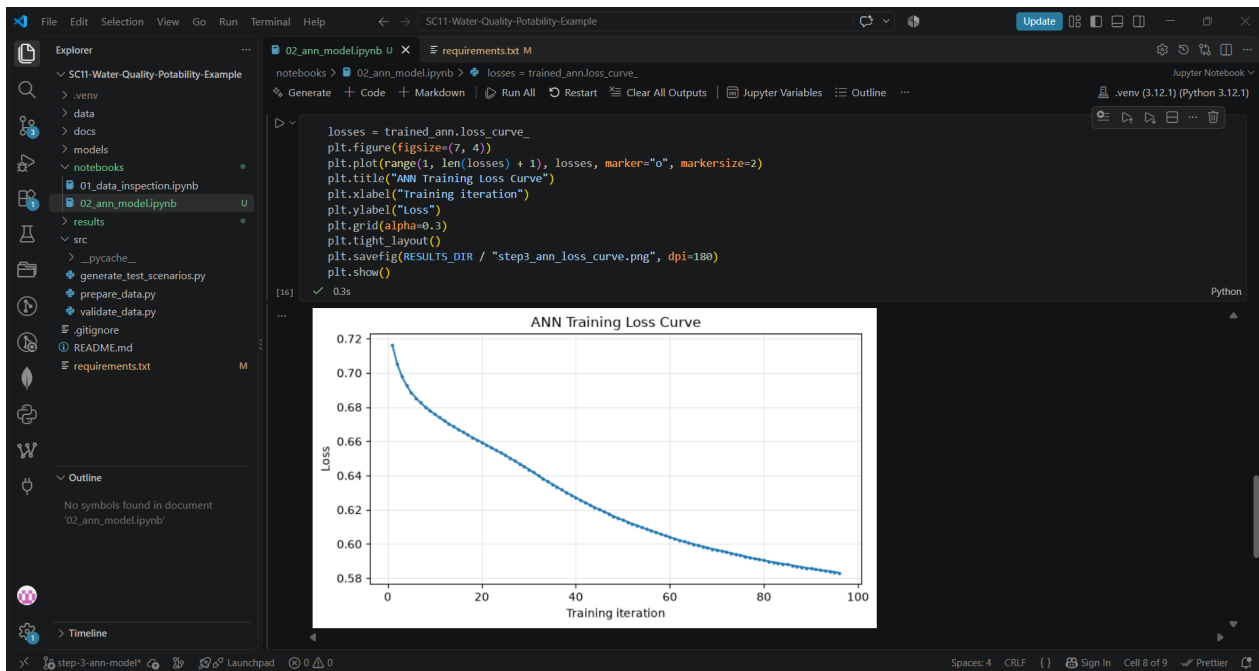
	split	accuracy	precision	recall	f1_score
0	validation	0.5443	0.5415	0.5781	0.5592

Screenshot 21. Validation metrics recorded in a table.

Action 22: Create and save the loss curve

Loss measures the model's error during training. The curve helps you see whether learning is progressing.

1. Create/adapt a loss-curve cell using the SC11 example as guidance, then run it for your own trained model.
2. It plots loss against training iteration and saves **results/step3_ann_loss_curve.png**.
3. A generally falling curve is expected. It does not by itself prove that the model is ready for real-world use.

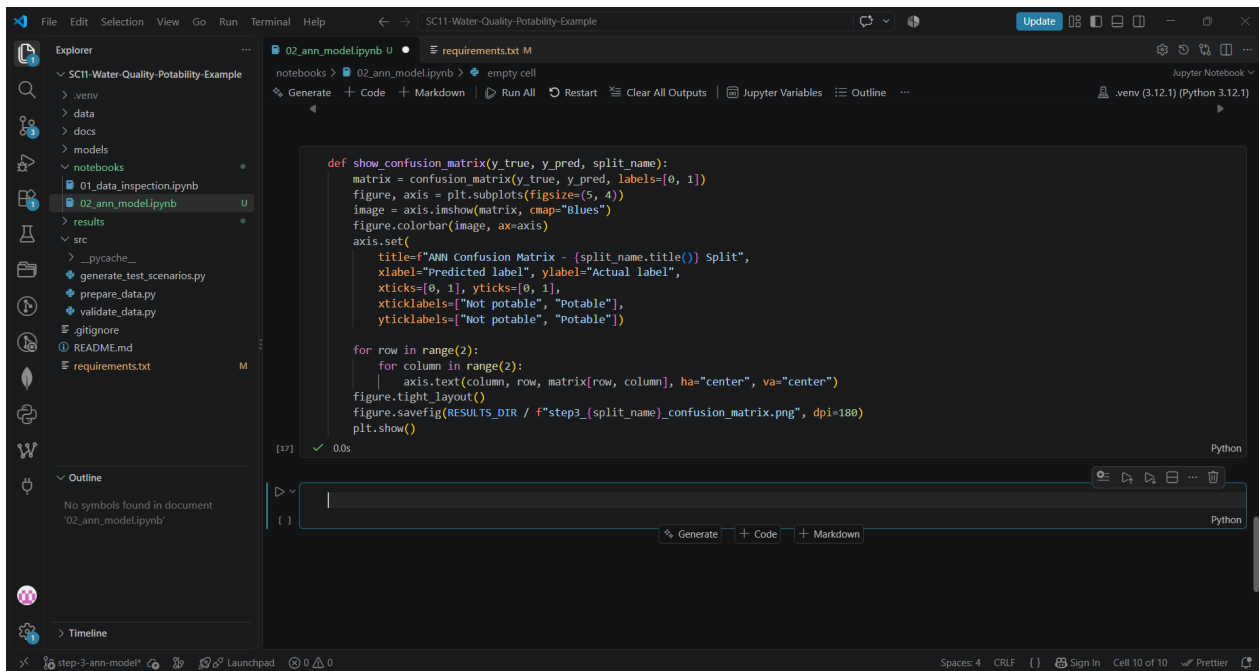


Screenshot 22. The decreasing ANN loss curve is both displayed and saved to results.

Action 23: Create the confusion-matrix function

A confusion matrix shows the number of correct and incorrect predictions for each class.

1. Use the SC11 `show_confusion_matrix` function as a reference and create/adapt an equivalent function for your own labels.
2. The function labels rows as actual values and columns as predicted values.
3. Run the cell before producing validation and test matrices.

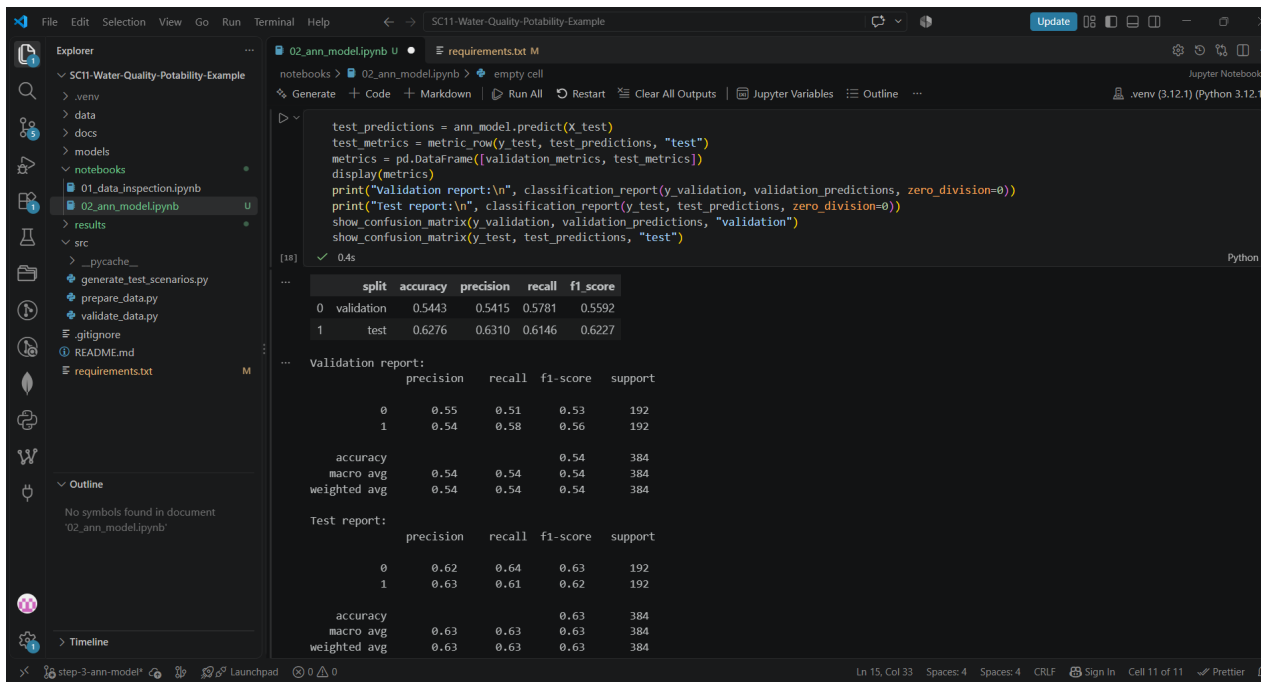


Screenshot 23. Function used to create and save a labelled confusion matrix.

Action 24: Evaluate with the final test split

The test split is kept separate until this final model check.

1. Run the test-evaluation cell after the validation work is complete.
2. Check that the results table contains both a validation row and a test row.
3. Review the printed class reports. These show performance for not potable and potable labels separately.
4. Do not repeatedly change settings until the test score improves; that would misuse the test split.



```
test_predictions = ann_model.predict(x_test)
test_metrics = metric_row(y_test, test_predictions, "test")
metrics = pd.DataFrame([validation_metrics, test_metrics])
display(metrics)

print("Validation report:\n", classification_report(y_validation, validation_predictions, zero_division=0))
print("Test report:\n", classification_report(y_test, test_predictions, zero_division=0))
show_confusion_matrix(y_validation, validation_predictions, "validation")
show_confusion_matrix(y_test, test_predictions, "test")
```

split	accuracy	precision	recall	f1_score
0 validation	0.5443	0.5415	0.5781	0.5592
1 test	0.6276	0.6310	0.6146	0.6227

Validation report:

	precision	recall	f1-score	support
0	0.55	0.51	0.53	192
1	0.54	0.58	0.56	192
accuracy			0.54	384
macro avg	0.54	0.54	0.54	384
weighted avg	0.54	0.54	0.54	384

Test report:

	precision	recall	f1-score	support
0	0.62	0.64	0.63	192
1	0.63	0.61	0.62	192
accuracy			0.63	384
macro avg	0.63	0.63	0.63	384
weighted avg	0.63	0.63	0.63	384

Screenshot 24. Validation and test metric rows plus detailed classification reports.

Action 25: Inspect validation and test confusion matrices

The matrix makes errors visible and supports a more thoughtful discussion than accuracy alone.

1. Read the top matrix as validation results and the lower matrix as final test results.
2. The diagonal cells are correct predictions. The off-diagonal cells are incorrect predictions.
3. Use the matrices in your report to explain where the ANN still needs improvement.

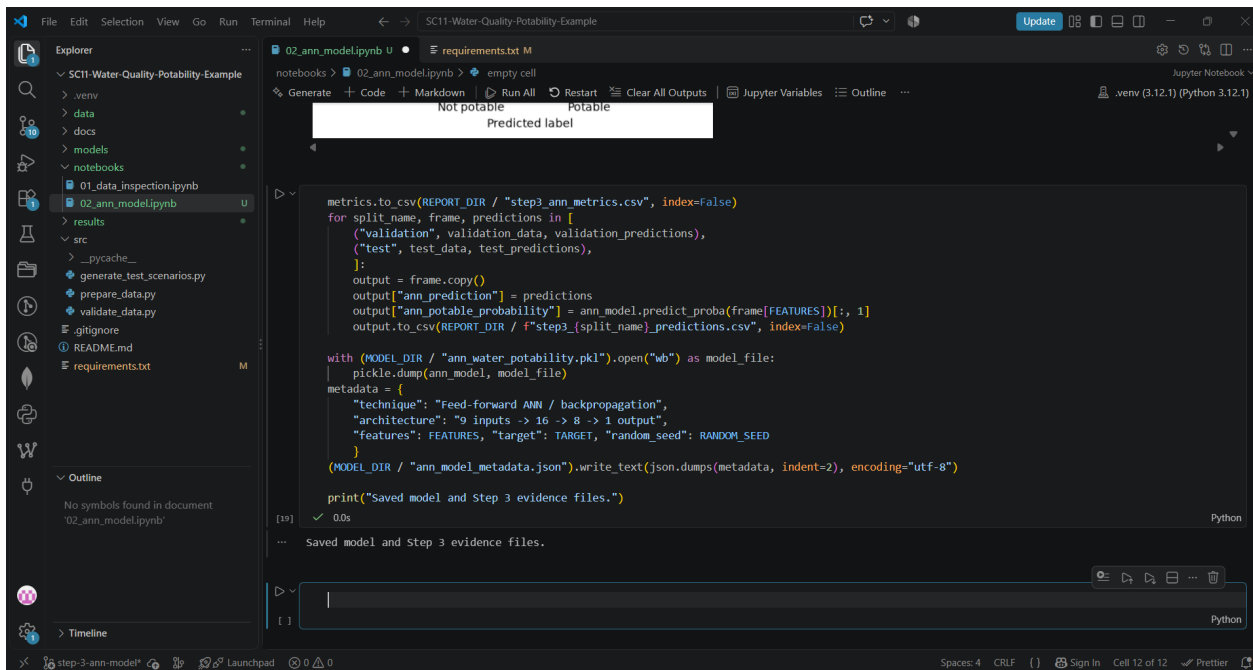


Screenshot 25. The notebook displays both validation and test confusion matrices.

Action 26: Save the model and all evidence

Saving prevents the work from existing only inside a temporary notebook session.

1. Run the final evidence-saving cell from the starter notebook.
2. It saves the metrics CSV, validation/test prediction files, model file, and metadata JSON.
3. Wait for the message **Saved model and Step 3 evidence files.** before moving on.

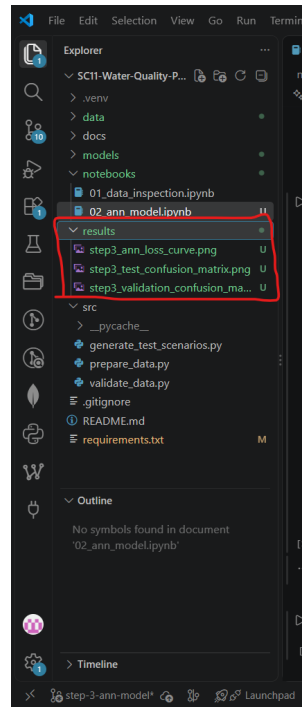


Screenshot 26. Successful completion message after model and evidence files are saved.

Action 27: Check the saved results folder

Use Explorer to confirm that the graph files were physically created in the repository.

1. Expand the **results** folder in Explorer.
2. Confirm the loss curve, test confusion matrix, and validation confusion matrix are present.
3. A green U means the new file is untracked and must still be added to Git later.

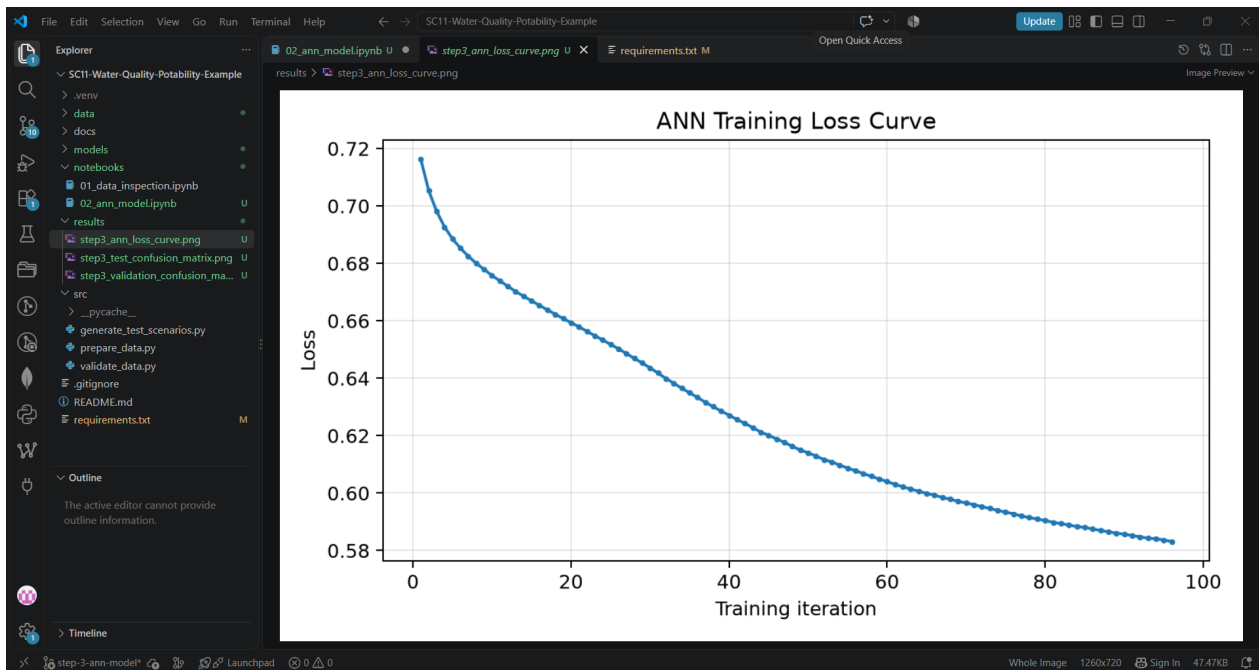


Screenshot 27. Explorer shows all three required Step 3 figure files.

Action 28: Open the saved evidence images

Opening the saved files confirms that they are usable outside the notebook.

1. Click the loss curve PNG in Explorer and verify that the title and axes are readable.
2. Open the test and validation confusion matrices too.
3. Use these images in the project report and presentation only after they are committed to Git.

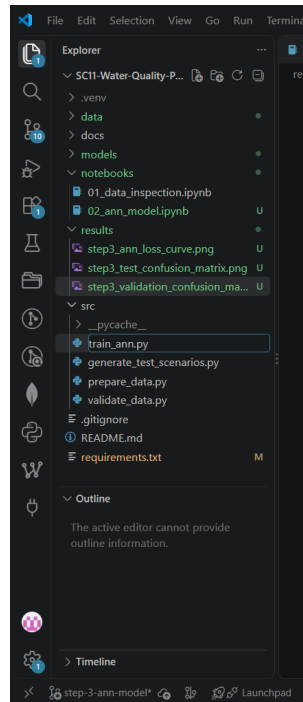


Screenshot 28. Saved ANN loss curve opened from the results folder.

Action 29: Create the reusable train_ann.py script

The Python script repeats the same process without relying on notebook cells.

1. Right-click the **src** folder and create a file named **train_ann.py**.
2. Copy the complete supplied train_ann.py code into it. Do not create a shortened version.
3. Save the file before running it.

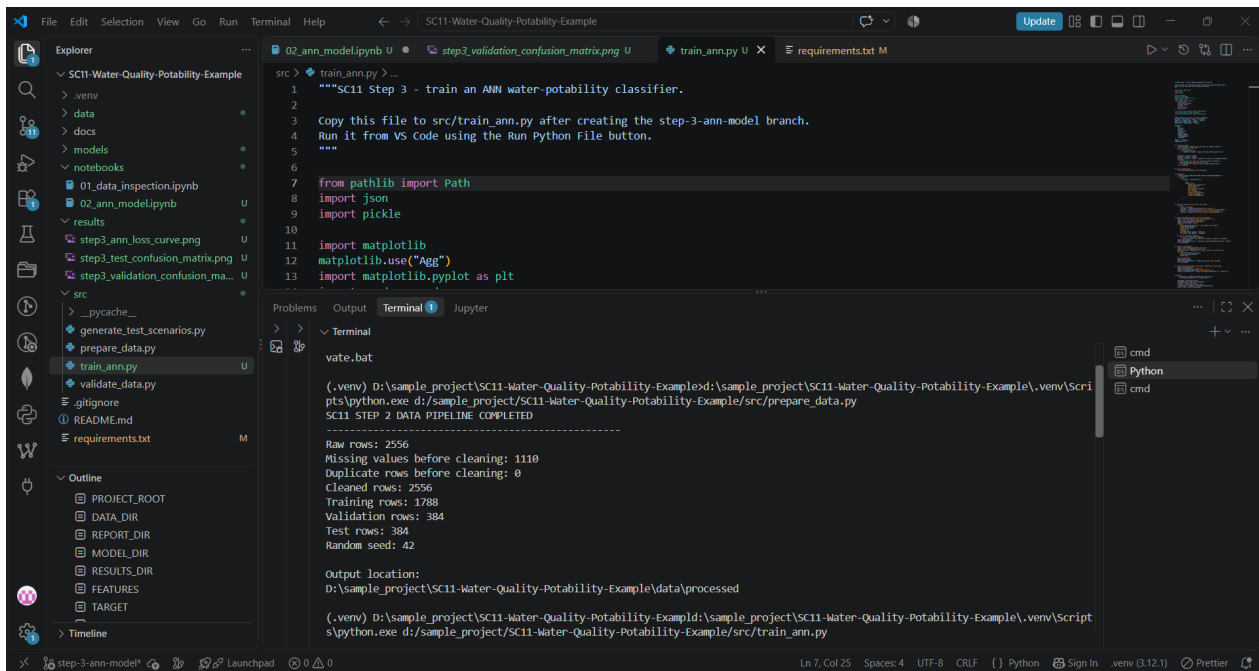


Screenshot 29. Create train_ann.py under src.

Action 30: Run the reusable Python script

The script makes the training process repeatable for a teammate or evaluator.

1. Open `train_ann.py` and use VS Code's Run Python File button at the top-right.
2. Wait for the output panel to complete. It should create the same model, metrics, prediction files, and figures.
3. If an error occurs, read the final error line, fix the named issue, and rerun only after saving the correction.

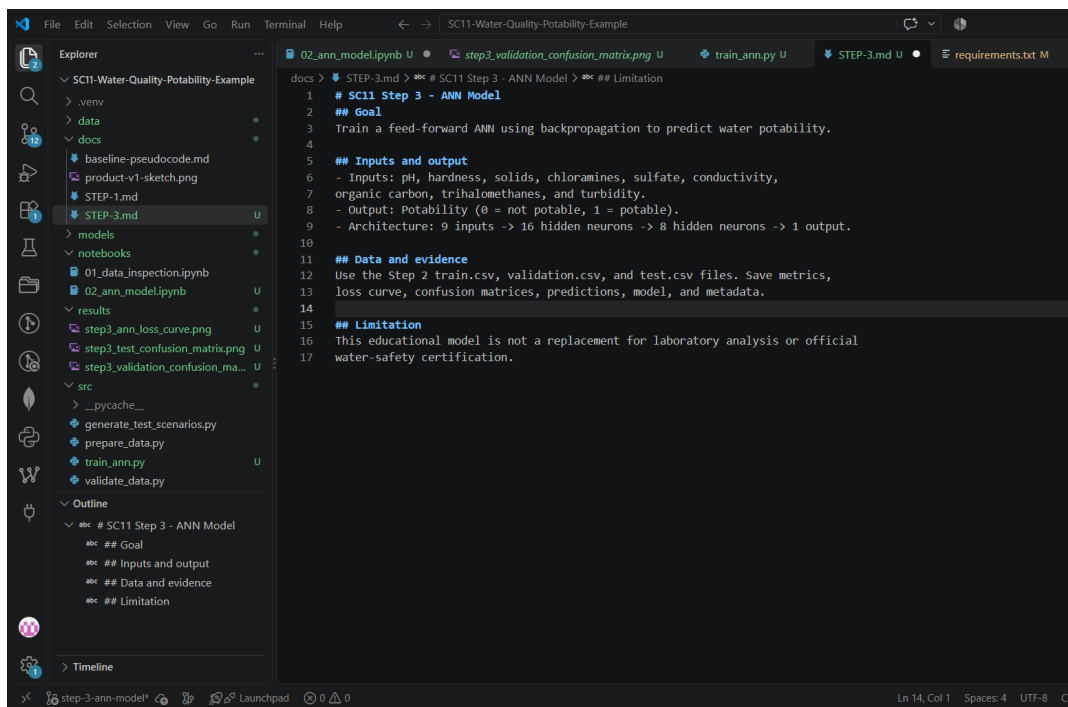


Screenshot 30. `train_ann.py` is placed under `src` and executed using the project environment.

Action 31: Create Step 3 documentation

A short Markdown document tells a reviewer what the ANN does without requiring them to inspect all source code.

1. Right-click the **docs** folder and create **STEP-3.md**.
2. Include the goal, nine inputs, output definition, ANN architecture, data strategy, evidence files, and limitation.
3. Use simple words and state clearly that the model does not replace laboratory testing.

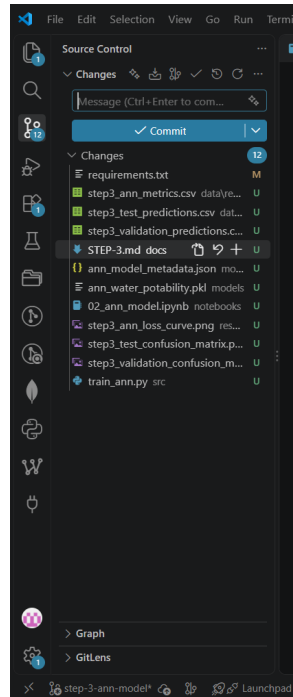


Screenshot 31. STEP-3.md documents the ANN goal, architecture, data, evidence, and limitation.

Action 32: Review changes before staging

Source Control shows every file that will become part of the Step 3 checkpoint.

1. Click the Source Control icon on the left.
2. Review the file list: notebook, script, documentation, requirements, model, CSVs, and figures should be present.
3. Do not commit accidental files such as personal notes, duplicate datasets, or unrelated screenshots.

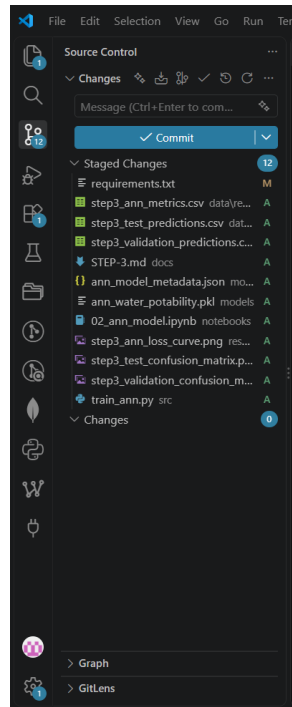


Screenshot 32. Source Control lists the complete Step 3 work before staging.

Action 33: Stage the reviewed files

Staging selects exactly which files will be included in the next Git history checkpoint.

1. After reviewing, click the plus icon beside each required file or the plus icon beside Changes.
2. Files move from Changes to **Staged Changes**.
3. Confirm that the Changes section is empty and the staged list contains all Step 3 evidence.

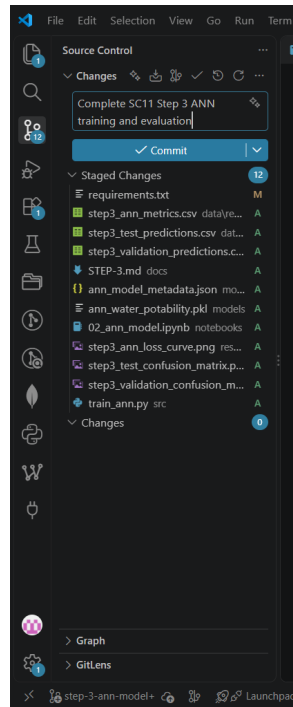


Screenshot 33. All reviewed Step 3 files are staged.

Action 34: Write a meaningful commit message

A commit message should state what was completed, not use vague words such as update or changes.

1. Click the message box at the top of Source Control.
2. Enter: **Complete SC11 Step 3 ANN training and evaluation.**
3. Click Commit only after the staged list is correct.

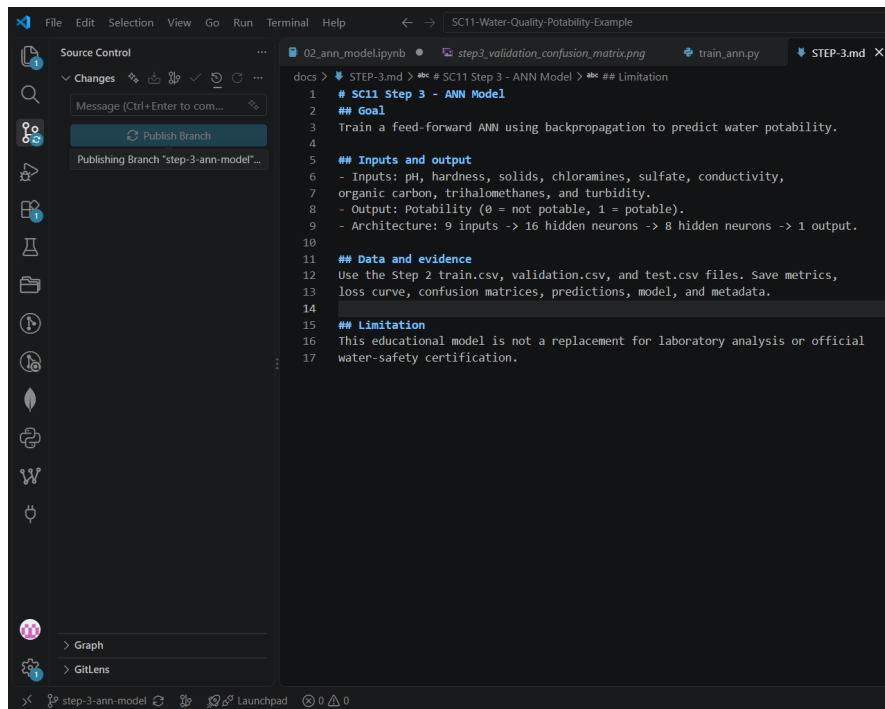


Screenshot 34. Clear commit message ready before pressing Commit.

Action 35: Publish the new branch

Publishing sends the local step-3-ann-model branch to GitHub for review.

1. After the commit finishes, click **Publish Branch** or **Sync Changes**.
2. Wait for the status message. Do not close VS Code while publishing is in progress.
3. Once complete, the branch is available on GitHub with its own commit history.

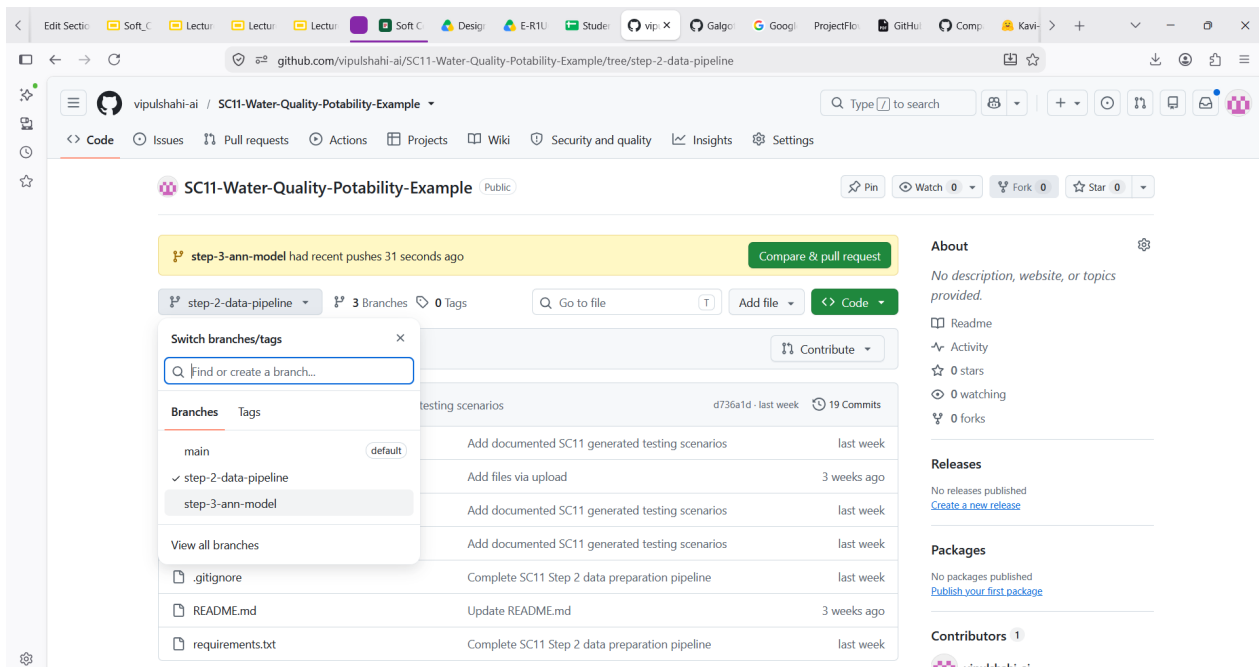


Screenshot 35. VS Code is publishing the step-3-ann-model branch.

Action 36: Verify the branch on GitHub

GitHub verification proves that the local work was successfully published.

1. Open the repository in a browser.
2. Use the branch drop-down and select **step-3-ann-model**.
3. Check that the most recent commit message matches your Step 3 commit.

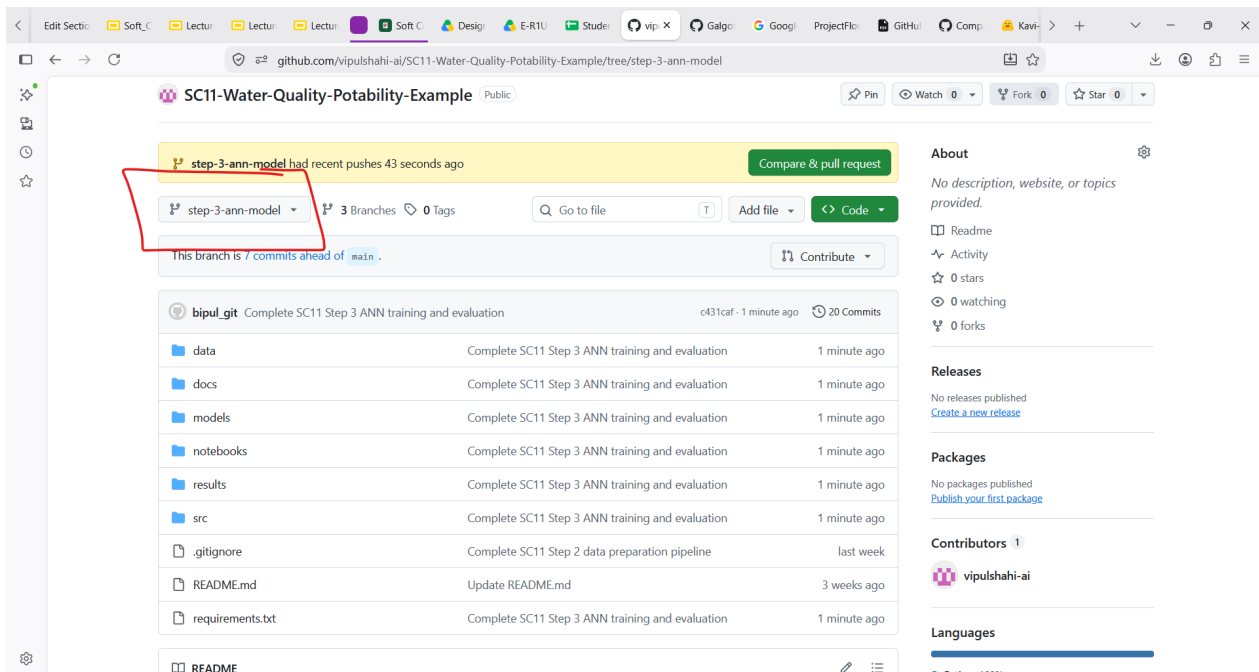


Screenshot 36. GitHub branch picker shows the new step-3-ann-model branch.

Action 37: Final GitHub check

This is the final evidence that Step 3 is complete and ready for review.

1. Check the branch name near the top-left of the GitHub file list.
2. Confirm that data, docs, models, notebooks, results, and src have Step 3 updates.
3. Keep this screenshot for your submission evidence. Do not merge into main unless instructed.



Screenshot 37. Step 3 branch, commit message, and all required folders are visible on GitHub.

Step 3 final checklist

1. I created step-3-ann-model from step-2-data-pipeline, not from main.
2. My notebook has executed outputs for data, model design, training, metrics, loss curve, and confusion matrices.
3. I installed and recorded required packages in requirements.txt.
4. I saved the model, metadata, metrics CSV, prediction CSVs, and images.
5. I created and ran src/train_ann.py.
6. I created docs/STEP-3.md with an honest limitation.
7. I staged, committed, published, and verified the work on GitHub.

What is next

Step 4 will create a baseline comparison so the team can discuss whether the ANN provides value compared with a simpler approach. Start Step 4 only after this branch is reviewed.